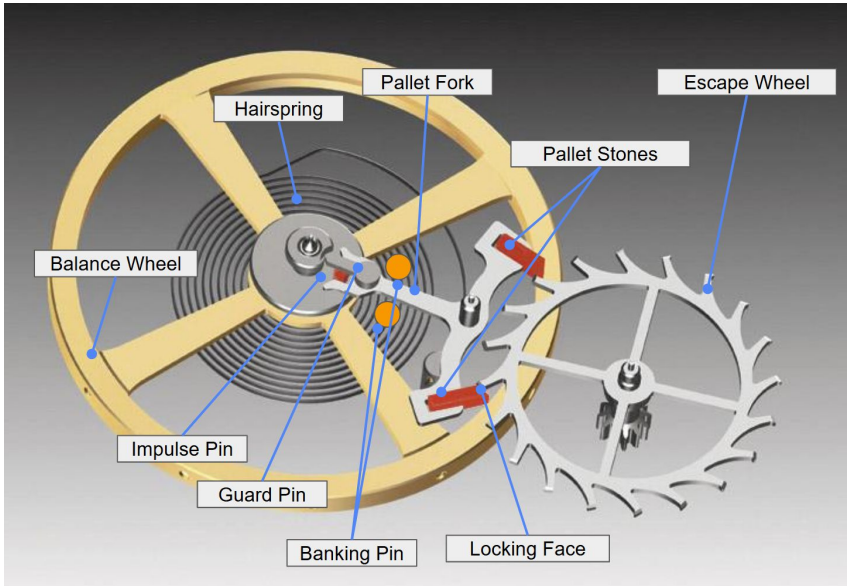
LG Software Architecture

Time Grapher
Kickoff Brief

From Tick to Trace: Real-Time Acoustic Analysis

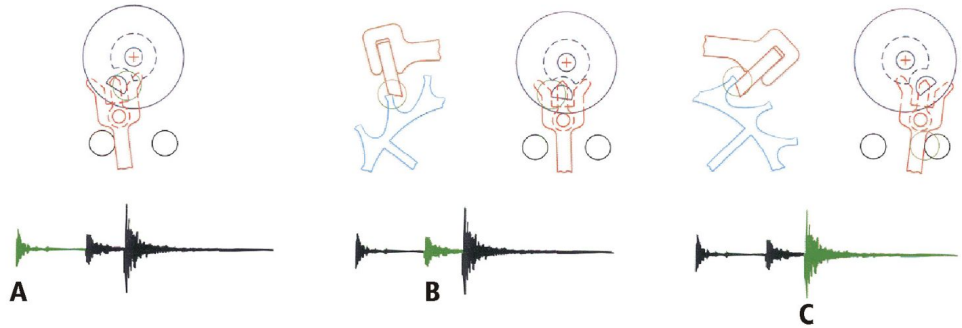
Overview of a Mechanical Watch



https://www.youtube.com/watch?v=9_QsCLYs2mY

Minute 4:12 to 6:30

All rate measurements and analysis of mechanical watches are based on the **acoustical signal generated by the escapement (beat noises)**



Produces 3 sounds per beat:

- First noise occurs when the impulse-pin of the roll strikes the fork of the pallets.
- Second noise occurs when a tooth on the escape wheel begins sliding across the impulse face of a pallet stone
- The third noise is created when a tooth of the escape-wheel meets the lockingplane of the pallet-stone and the lever hits the banking-pin.

Measuring Acoustic Signal

We can observe, interpret, and diagnose a mechanical watch from its acoustic signal

With machines, we can graph the watch behavior

- Functions as a diagnostic aid, not just a graphing tool
- Helps determine:
  - whether the watch is running accurately
  - whether the beat pattern is stable
  - whether amplitude is healthy
  - whether there are signs of mechanical problems

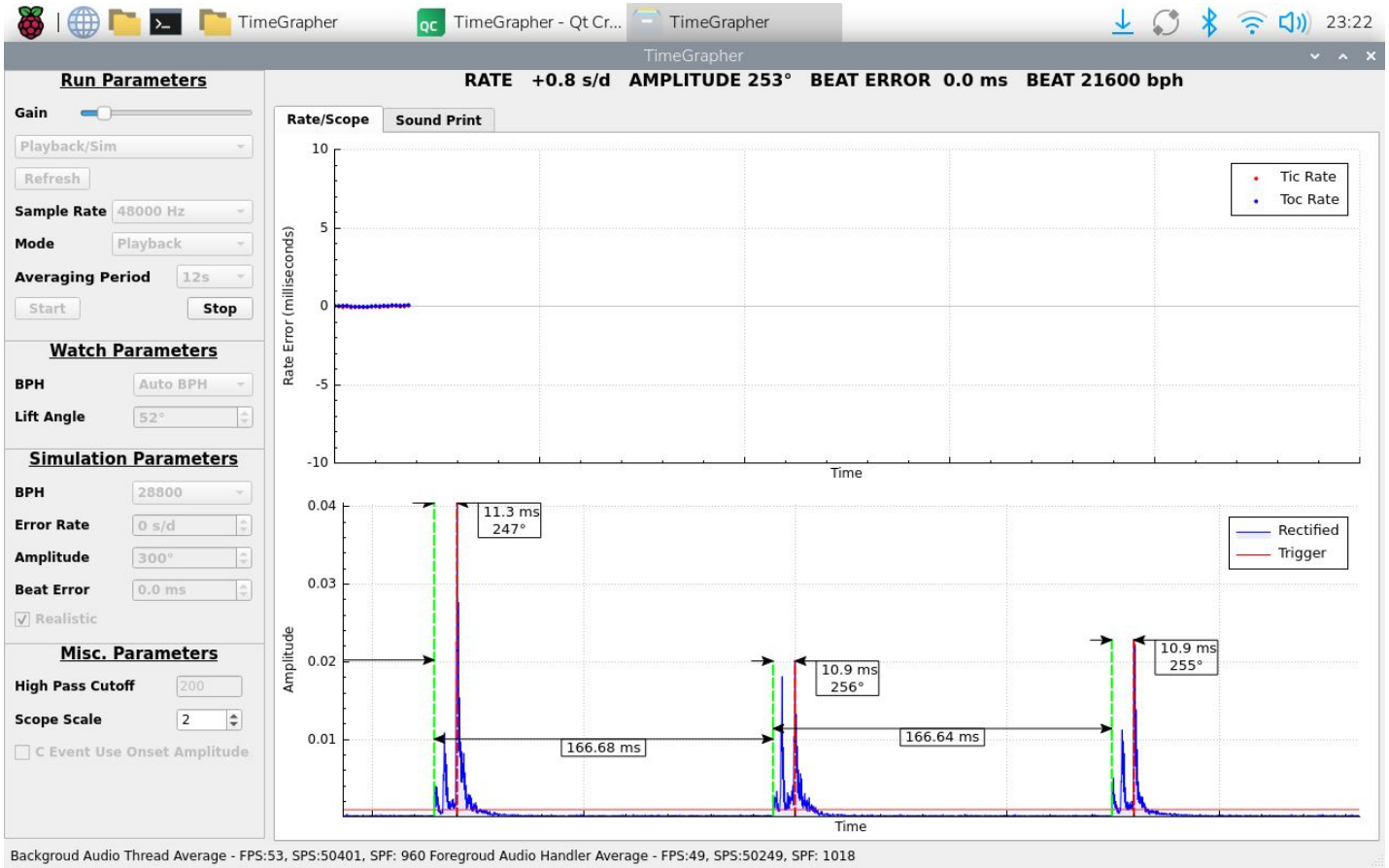
We can enhance the graphs with labels, markers, alerts, interactive features, derived measures, and explanatory text to make results easy to understand in real time

We can improve usability through filtering and signal clarity

- Reduce extraneous noise while preserving features needed for correct detection and analysis

Let’s see a demonstration!

Demonstration



Overview of the Project

Advance and improve the core functions of the baseline mechanical-watch timegrapher as a real-time diagnostic and visualization system capable of running on a Raspberry Pi.

- Signal detection
- Signal processing
- Real-time analysis
- Real-time graphing

Modify and improve the remote user interface application

- Improve the control settings and parameters within the remote user interface application
- Add the additional mandatory graphs (defined in the project plan)

Add on-device intelligence running lightweight AI or TinyML models directly on the Raspberry Pi

Consider the architectural drivers and quality attributes of the system

- Redesign and refactor as needed to make intentional tradeoffs

Demonstrate team's innovation and creativity during milestone reviews and a final demonstration

The "as-is" demonstration model, product, and code that your team will receive functions, but Solvit Inc believes the software architecture may be significantly improved.

LG SA Development

Mandatory Graphs

- Trace Display
- Rate and Amplitude Stability Over Time
- Multi-Position Sequence Display
- Beat-Noise Scope Display
- Beat Error Display and Diagnostic Trace
- Long-Term Performance Graph
- Escapement Analyzer and Marker-Line Display
- Time-Frequency Spectrogram Display
- Waveform Comparison Display with Timing Markers
- Scope Mode with Synchronized Sweep Display
- Scope Function with Multiple Filter Views

Enhanced Features or Graphs

Additional Acoustic Measures

- See Chour reference for examples

User Interface Functionality

- All graphs can run continuously
- Interactive controls such as start, stop, pause
- Ability to move backward and forward in time through captured data
- Ability to pause the display and inspect prior data without losing the recorded signal or forcing a reset
- Support interactive selection of timing points and measurement regions (e.g., show 6 beats/sec)

Enhance the existing sound graph

AI Feature (examples include)

- Signal Quality Classification
- Bad Data Rejection
- Fast/Slow Watch Classification
- User Guidance

Overview of Time Grapher System

Remote User Interface

- Displays rate/scope and sound print views
- Displays control panel to adjust run time parameters
- Allows user to toggle between microphone, playback, or simulation

USB Watch Stand and Sensor

- Holds watch in position
- Measures analog watch signal (acoustic signal)
- Transforms analog signal into digital signal
- USB interface to support computer or Raspberry Pi

WeiShi 1000 Time Grapher with Watch Stand / Sensor

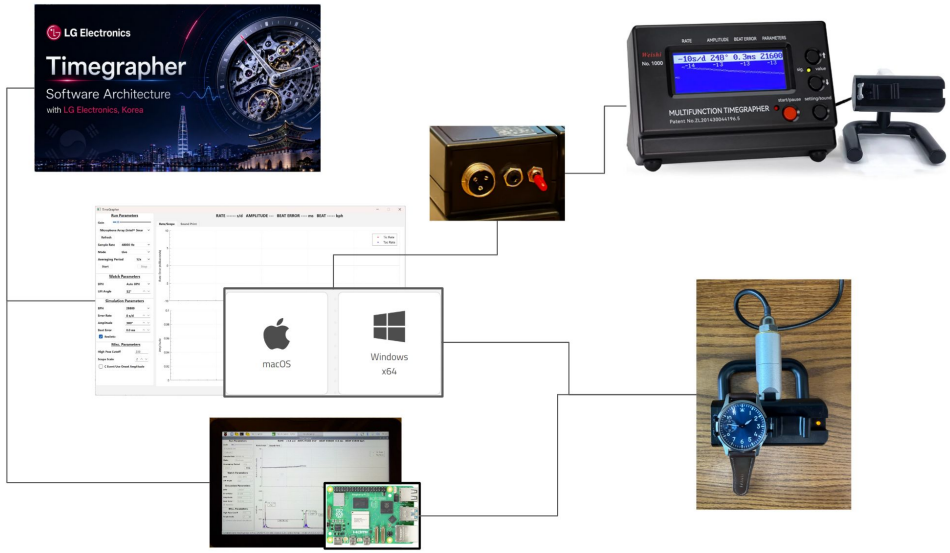
- Allows teams to compare measurements with GUI

Raspberry Pi with Touchscreen

- Embedded device capable of running the TimeGrapher application

Converter Box

- Allows users to use a watch sensor with a different cable plug



As-Is Provided Software Architecture and Demo Code

Overview of Play Modes

Live

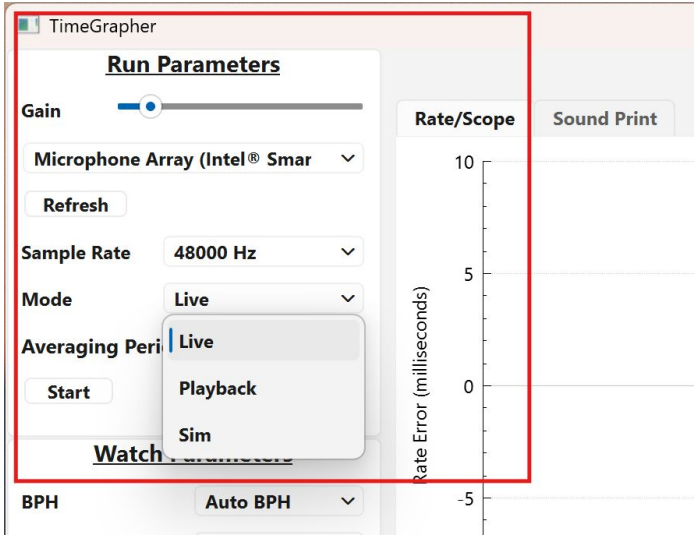
- Captures and analyzes signal data directly from the microphone in real time.

Playback

- Uses a previously recorded signal instead of live microphone input.

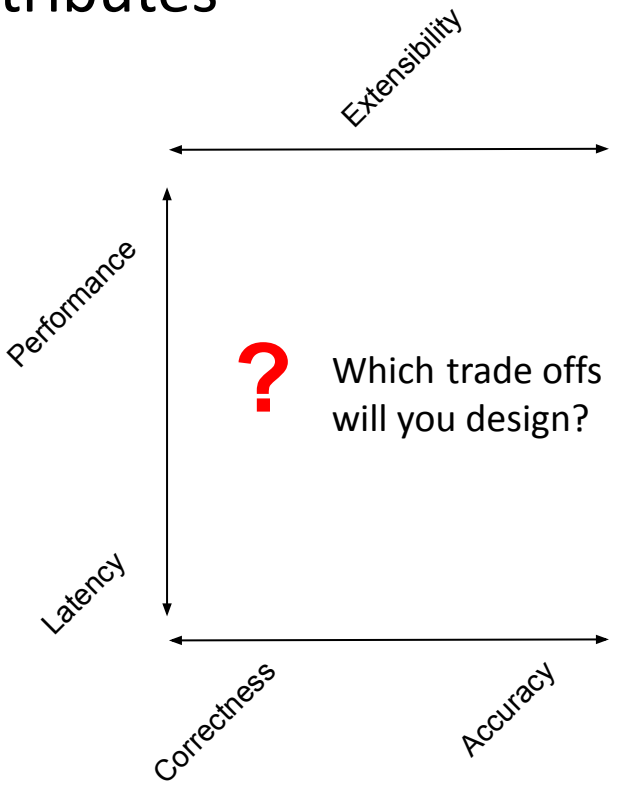
Sim

- Generates a synthetic watch signal for testing and development.



Desired TimeGrapher System Quality Attributes

- Real Time Performance: The system shall acquire, process, analyze, and display watch acoustic data in real time.
- Low Latency and Low Number of Missed Beats: The system shall minimize end-to-end latency between acoustic capture at the microphone and presentation of the corresponding waveform, markers, and computed values in the GUI.
- Correctness: The system shall compute watch-performance measures accurately and consistently, with displayed values and graphs that remain aligned with the underlying events across the GUI and summaries.
- Measurement Accuracy, Error Detection, and Handling: The system shall detect the relevant watch events with sufficient accuracy to support meaningful measurement of small timing differences.
- Extensibility, Modifiability: The system shall be easy to understand, extend, test, and debug within the limited project schedule.



System Requirements of the TimeGrapher System

- Ensure resilient and reliable operation of the Raspberry Pi–based TimeGrapher during signal capture, processing, and display.
- Capture, process, and analyze mechanical-watch acoustic signals in real time.
- Minimize latency in acquisition, event detection, calculation, and graphing.
- Provide an interactive graphical user interface for live signals, playback, simulation, and recorded data review.
- Support the addition and replacement of measurements, graphs, filters, and analysis features that provide insight into watch health, stability, accuracy, and mechanical condition.
- Provide accurate and consistent computation of watch-performance measures and ensure that displayed values correspond to the underlying acoustic events.
- Support filtering and signal conditioning to improve robustness in the presence of noise, weak signals, or corrupted data.
- Provide architecture that is modular, extensible, maintainable, and testable.
- Implement, where feasible, a proof of concept for on-device AI or TinyML on the Raspberry Pi to support future signal-analysis and user-guidance features.

Some administrative points before you embark…

Goals of the Project

- Understand the influence of architectural drivers on software structures.
- Understand the technical, organizational, and business role of software architecture.
- Recognize the importance of quality attributes, document quality attribute requirements, and choose among tactics and patterns that help to realize requirements of performance, accuracy, security, extensibility, etc.
- Identify fundamental architecture structures (modules, components and connectors, deployment, etc.) and key architectural approaches (styles, patterns, tactics, etc.).
- Understand the principles of good architecture documentation and presentation.
- Learn how to document an architecture using multiple views, using informal notations and UML, complementing structural diagrams with behavior diagrams, and recording design decisions.

Schedule - Key Events

Week	Dates	Event	Event / Due Date
Week 0	05/25-05/29	Kick Off meeting (Project Introduction, Team assignment, Equipment Issue)	Wednesday May 27
Week 1	06/01/06/05		
Week 2	06/08-06/12	Milestone 1 - Project Plan, Architectural Drivers, Risk Assessment / Planned Experiments, Architectural Approaches	Tuesday June 09
Week 3	06/15-06/19		
Week 4	06/22-06/26	Milestone 2- Project Plan, Experiment Results, Architecture	Monday June 22
Week 5	06/29-07/01	Milestone 3 - Final Demonstration	Wednesday July 01

TimeGrapher System Information & Materials

- The following is posted to Canvas:
  - TimeGrapher_v10.5_Student.zip
  - Time Grapher Project Plan
  - TimeGrapher GUI Set Up Instructions
  - TimeGrapher Equations_v0
  - Witschi-Training-Course
  - Witschi Chronoscope X1 G3 Instruction Manual
- Assembled Raspberry Pi 5 with Touchscreen:
  - TimeGrapher_v10.5_Student.zip on Desktop
  - Qt Creator .run file on Desktop
- WeiShi Timegrapher with Watch Holder / Sensor Stand
- Seperate USB capable Watch Holder / Sensor Stand
- Solvit, Inc Converter Box
- Two Watches

Project Questions

Engineering and design questions:

- ❏ Refer to your Paulo and Matt

Project requirement questions:

- ❏ Please email your questions to Jason Popowski (jpopowsk@andrew.cmu.edu) and Steve Beck (srbeck@andrew.cmu.edu)
- ❏ Ensure to courtesy copy Dan Plakosh (dplakosh@sei.cmu.edu)

Remember your three deliverables / milestones (reference Project Plan):

- ❏ Requirements, project plan, plan for experimentation, and risks
- ❏ Experimentation results, design, plan for construction
- ❏ Demo and lessons learned

Final Comments and Hints

Read the Project Plan in its entirety

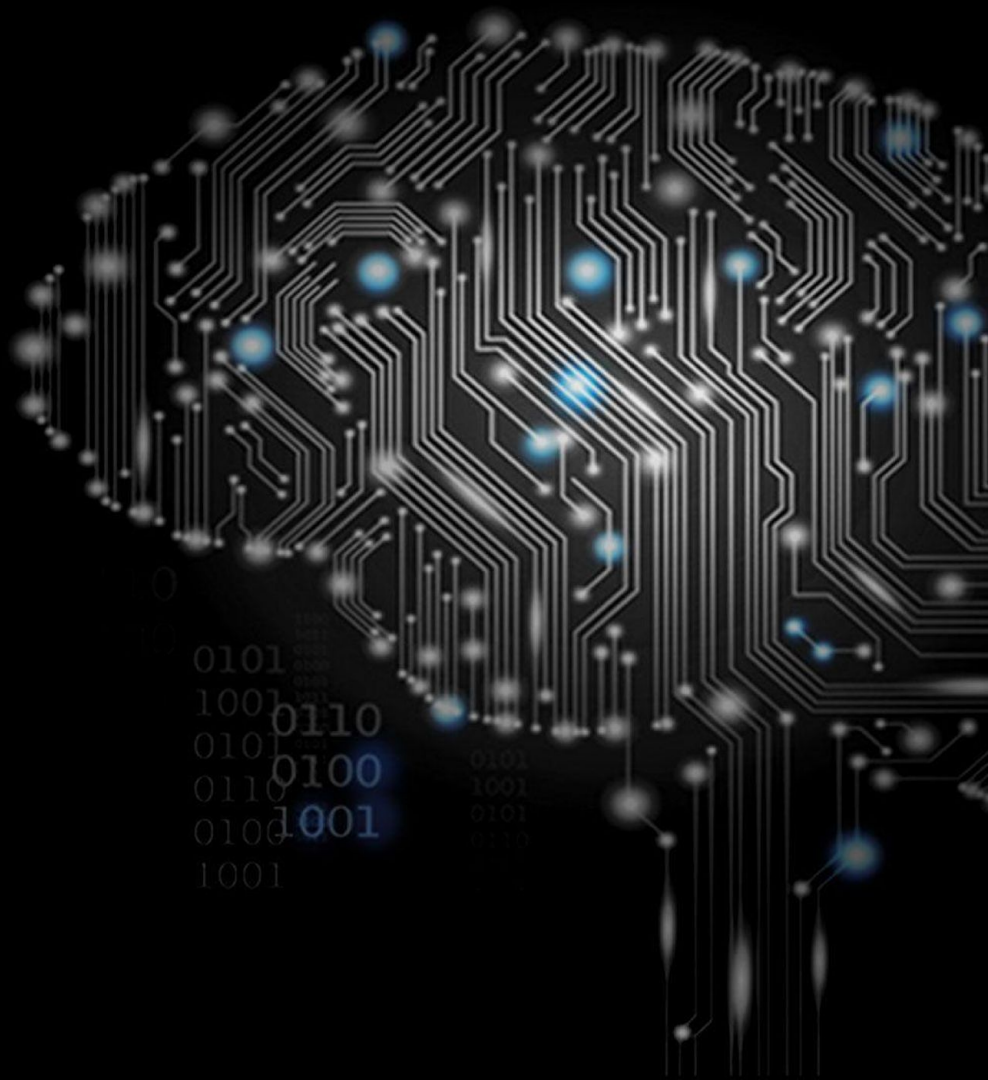
Know the code and how it actually works

Look for better ways to design or implement functionality and quality attributes then the source code provided

A few cautions:

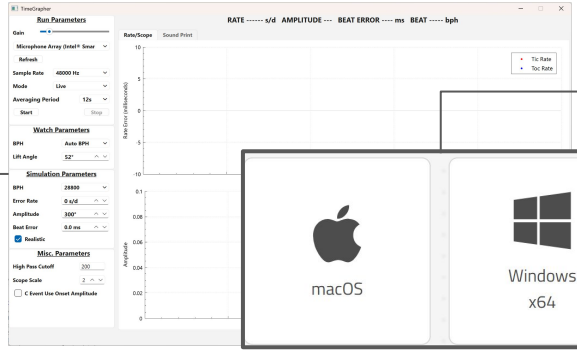
- Not every issue in this project will be caused by software.
- Variations in watch condition, sensor placement, microphone coupling, and ambient noise can all influence the measured results.
- A solution that performs well on a PC may still require optimization to achieve the project's real-time, latency, and performance goals on the Raspberry Pi.
- Teams should therefore test and evaluate their design on the target hardware, not only on their development machines.

Questions?




LG Electronics
Timegrapher
Software Architecture
with LG Electronics, Korea


RATE
AMPLITUDE
BEAT ERROR
PARAMETERS
Weishi
No. 1000
-10s/d 248° 0.3ms 21600
sig.
value
start/pause
setting/sound
MULTIFUNCTION TIMEGRAPHER
Patent No.ZL201430044196.5


RATE ------ s/d AMPLITUDE --- BEAT ERROR ---- ms BEAT ----- bph
Run Parameters
Watch Parameters
Simulation Parameters
Misc. Parameters

macOS
Windows
x64